Parcial práctico 2 de backend

Programación Web | NestJS y TypeORM

Modalidad: individual. **Evaluación:** código y funcionamiento de la solución.

Completar un backend existente para gestionar reservas de salas. Se deben desarrollar **dos endpoints y tres pruebas unitarias**. El proyecto se entregará configurado; no es necesario comenzar desde cero ni rehacer un CRUD completo.

1 Caso

Una organización registra reservas para utilizar sus salas de reunión. Cada reserva pertenece a una sala existente y se crea con estado `active`. Cuando se cancela, su estado cambia a `cancelled`. Una reserva que ya está cancelada no puede volver a cancelarse.

El ejercicio no maneja fechas, cruces de horarios, usuarios ni cobros. Cancelar no elimina el registro, no modifica la sala y no cambia la cantidad de participantes.

Entidad	Datos ya definidos
`RoomEntity`	`id`, `name` y relación `reservations`.
`ReservationEntity`	`id`, `purpose`, `participants`, `status` y relación `room`.

Una sala puede tener varias reservas; cada reserva pertenece a una sola sala. Las entidades, sus relaciones y los estados `active` y `cancelled` ya estarán definidos en el proyecto base.

2 Punto de partida

El docente entregará una carpeta con NestJS, TypeORM, PostgreSQL, Docker Compose y Jest configurados. Se debe trabajar sobre esa carpeta y seguir su README para ejecutar la API y las pruebas.

- Las consultas `GET /api/rooms`, `GET /api/reservations` y `GET /api/reservations/:id` estarán terminadas y habrá datos de ejemplo con reservas activas y canceladas.
- `ReservationsService.create()` ya buscará la sala, rechazará una sala inexistente con `404`, asignará `active`, relacionará la reserva y la guardará.
- `ReservationsService.findOne(id)` ya consultará la reserva con su sala y lanzará `NotFoundException` cuando no exista.
- El pipe compartido `requestValidationPipe` y los imports necesarios estarán preparados. Los dos archivos de pruebas incluirán `describe`, `beforeEach`, datos y mocks; faltará completar tres `it`.

Archivos de trabajo: `create-reservation.dto.ts`, `reservations.controller.ts`, `reservations.service.ts`, `reservation-rules.service.ts`, `reservations.module.ts` y los dos archivos `.spec.ts` de servicios. No modificar la configuración, las entidades ni los métodos ya resueltos.

3 Crear una reserva

Endpoint: `POST /api/reservations`. Completar `CreateReservationDto` y el método del controlador. Delegar en `ReservationsService.create()`, que ya estará implementado.

Campo del body	Condición obligatoria
`purpose`	Texto obligatorio, diferente de "", con máximo 80 caracteres.
`participants`	Número entero obligatorio entre 1 y 20, incluidos ambos límites.
`roomId`	Número entero obligatorio mayor o igual a 1.

Aplicar `requestValidationPipe` al body y rechazar propiedades adicionales. Los números deben enviarse como números JSON; `"3"` no es válido en lugar de `3`. El cliente no puede enviar `status`: el servicio lo asigna. No crear otro pipe ni agregar validaciones dentro del controlador.

Ejemplo de body válido. Reemplazar `roomId` por el ID real de una sala existente:

```
{ "purpose": "Project planning", "participants": 6, "roomId": 1 }
```

Resultados esperados: `201` con la reserva, su ID, estado `active` y sala relacionada; `400` si la entrada incumple el DTO o incluye propiedades adicionales; `404` si el ID de sala es válido, pero no existe. La comprobación de existencia y el guardado ya corresponden al servicio proporcionado.

4 Cancelar una reserva

Endpoint: `PATCH /api/reservations/:id/cancel`. No recibe body. Completar el controlador, `ReservationRulesService.ensureCanCancel(reservation)` y `ReservationsService.cancel(id)`.

Regla de negocio

`ensureCanCancel()` debe permitir una reserva con estado `active` y lanzar `ConflictException` si ya está `cancelled`. Este servicio de reglas recibe la reserva; no consulta repositorios, no cambia el estado y no guarda.

Operación y dependencias

El controlador debe recibir el ID con `ParseIntPipe` y delegar. En `ReservationsService.cancel(id)`, utilizar el `findOne()` ya resuelto, llamar al servicio de reglas **antes de modificar**, cambiar el estado a `cancelled`, guardar con el repositorio y devolver su resultado. Conservar los demás atributos y la relación con la sala.

Registrar `ReservationRulesService` como proveedor en `ReservationsModule` e inyectarlo por constructor en `ReservationsService`. Debe tener `@Injectable()`; no crearlo con `new` dentro del servicio principal ni repetir la regla en el controlador.

Situación	Respuesta y efecto esperado
Reserva activa existente	`200`; queda en `cancelled`. Un GET posterior confirma el cambio.
Reserva ya cancelada	`409`; no se modifica ni se llama a `save()`.
ID como `abc`	`400`, antes de ejecutar el método del servicio.
ID entero de una reserva inexistente	`404`, usando el comportamiento de `findOne()`.

Enviar las peticiones una a la vez. No se exigen transacciones ni manejo de solicitudes simultáneas. No basta con devolver un objeto con otro estado: el cambio debe persistir.

5 Completar tres pruebas unitarias

Completar únicamente los tres `it` proporcionados y conservar los nombres en inglés. Utilizar el servicio real que se evalúa en cada caso; simular solo sus dependencias. No importar `AppModule` ni conectar los tests a PostgreSQL.

Prueba 1 La regla rechaza una reserva cancelada

En `reservation-rules.service.spec.ts`, utilizar el `ReservationRulesService` real y una reserva con `status: "cancelled"`. Comprobar que `ensureCanCancel()` lanza `ConflictException`.

Prueba 2 El servicio cancela y guarda

En `reservations.service.spec.ts`, preparar el mock del repositorio para devolver una reserva activa. La regla simulada permite continuar y `save()` devuelve la reserva cancelada. Ejecutar `cancel(id)` y comprobar que se llama al servicio de reglas, se solicita guardar con estado `cancelled` y se devuelve el resultado de `save()`.

Prueba 3 Si la regla rechaza no se modifica ni se guarda

Preparar una reserva con `status: "active"` y hacer que el mock de `ensureCanCancel()` lance `ConflictException`. Comprobar que `cancel(id)` rechaza con esa excepción, la reserva conserva `active` y `save()` no es llamado.

Importante: en la tercera prueba se simula deliberadamente un rechazo sobre una reserva activa. Se evalúa que el servicio respete a su dependencia antes de modificar los datos; no se está ejecutando la regla real en ese caso.

Ejecutar `npm test`. Deben pasar **exactamente tres pruebas**. No dejar casos vacíos ni usar `skip`, `todo` u `only`. Una comprobación de `toBeDefined()` no sustituye los comportamientos solicitados. Los errores iniciales `TODO TEST` se reemplazan por las comprobaciones, no se ocultan.

6 Comprobación y entrega

Verificar también la API en Postman: los tests de servicios no ejecutan los validadores ni los pipes del controlador. Como mínimo, comprobar creación válida, número enviado como texto, propiedad adicional, sala inexistente, cancelación exitosa, segundo intento de cancelación, ID inválido e ID inexistente.

Entregar el enlace al repositorio y la rama utilizada, con el código actualizado y commits que muestren el avance durante la evaluación. Incluir un archivo `exam-results.md` breve con las peticiones realizadas, los códigos HTTP obtenidos y el resumen de `npm test`.

No incluir `node_modules`, `dist`, archivos `.env` privados ni credenciales reales. No se solicitan preguntas teóricas, frontend, autenticación, autorización, JWT, Passport, despliegue, filtros de búsqueda, nuevos endpoints para salas ni un CRUD completo.

Se evaluarán el funcionamiento de los dos endpoints, los DTO y pipes, la separación de responsabilidades, el registro e inyección del servicio de reglas, la persistencia con TypeORM y las tres pruebas. La calificación corresponde al código entregado; no hay un componente separado de sustentación oral.

7 Restricciones y condiciones

1. Se permite consultar los apuntes personales del curso, la documentación del curso y enlaces relacionados con las tecnologías del ejercicio.

2. Se permite utilizar el navegador para consultar documentación y realizar búsquedas específicas en documentación oficial o foros técnicos. Este permiso no incluye consultar respuestas generadas por IA ni soluciones completas del parcial.

3. No está permitido utilizar ninguna herramienta de inteligencia artificial generativa durante la evaluación, ni para desarrollar, corregir, explicar o completar el ejercicio, ni para resolver dudas. Esto incluye ChatGPT, Gemini, Copilot, Claude y herramientas similares, aunque estén integradas en el navegador o el editor.

4. Antes de comenzar, desactivar asistentes, extensiones y autocompletado generativo de IA. Se permiten el autocompletado convencional del editor, el compilador, el depurador, Postman y Jest.

5. Solo se puede utilizar el computador personal o el suministrado por el laboratorio y autorizado para la evaluación. No se permite usar teléfonos, tabletas, relojes inteligentes ni otros dispositivos electrónicos adicionales.

6. La evaluación es individual. No se permite compartir código, respuestas, archivos o soluciones, comunicarse con otras personas para resolver el ejercicio ni permitir que alguien más lo realice.

7. Se debe trabajar sobre el proyecto base entregado. No se permite copiar un proyecto externo, incorporar una solución completa previamente preparada ni utilizar código desarrollado por otra persona como solución del parcial. Los ejemplos de los apuntes son material de consulta, no una entrega para copiar y pegar.

8. Se deben utilizar NestJS y TypeORM. No sustituir los repositorios por arreglos en memoria, respuestas fijas o simulaciones dentro de la aplicación para aparentar que funciona. Los mocks se utilizan únicamente en las pruebas.

9. Las entidades, atributos, variables, métodos, archivos, rutas y nombres de los tests deben estar escritos en inglés. Mantener los nombres y la estructura solicitados.

10. Realizar commits frecuentes que evidencien el desarrollo durante la evaluación y cuando el docente lo solicite. Sus mensajes deben describir los cambios. El repositorio y la rama entregada deben estar actualizados antes de la hora límite indicada por el docente.

11. No eliminar pruebas, desactivar validaciones, alterar la configuración para ocultar errores ni modificar los métodos ya resueltos para evitar los requisitos del enunciado.

12. Informar inmediatamente cualquier inconveniente técnico al docente. El historial de commits y el código podrán revisarse para verificar autoría y funcionamiento; un único commit final no demuestra por sí solo el avance solicitado.

El uso indebido de herramientas o recursos, incluido el uso de IA generativa, podrá ocasionar la anulación de la evaluación y el reporte correspondiente para el trámite institucional que aplique.